

01 — Criar um ambiente: a camada Docker

Como o container de um ambiente é criado no nó. Todo o trabalho com o Docker daemon acontece no **agente** (`apps/agent/src/docker.ts`, via `dockerode`), chamado pela API por HTTP sobre WireGuard. A API decide **o quê**; o agente faz **como**.

Há três formas de ambiente, com funções distintas no agente:

Forma	Função no agente	Exemplos
app (código)	<code>provision()</code> — <code>docker.ts:623-725</code>	php, node, python, dotnet, static
service (banco/serviço)	<code>provisionService()</code> — <code>docker.ts:819-873</code>	redis, mysql, mariadb, postgres, mongodb, rabbitmq
stack (app + banco-filho)	orquestra 2× <code>provisionService</code> — <code>provisioner.ts:116-162</code>	n8n, wordpress

1. App de código — `provision()`

Ordem (`docker.ts:623-694`):

1. `resolveImage(runtime) → ensureImage()` (§4).
2. `removeExistingByEnv(envId)` — remove qualquer container com label `vp.env=<envId>` (idempotência em `retry`).
3. `lxcfsBinds()` (§6) — binds do LXCFS.
4. Volume de código `veloz-code-<envId>` só para `python/dotnet` (`-/app`) e `static` (`-/site`) (§5).
5. `pickCpuset(vcpu)` (§6).
6. `ensureNetwork(...)` — bridge do dono (ver [02 \(02-ambiente-rede.md\)](#)).
7. `createContainer(...)` → `start()`.

Objeto real do `createContainer` (`docker.ts:648-692`)

```
{
  Image: image,
  ... (runtime.kind === "static" ? { Entrypoint: ["/bin/sh"] } : {}), // caddy tem ENTRYPOINT próprio
  Cmd: cmdFor(runtime, args.startupScript), // o SUPERVISOR do runtime (§7)
  Env: [
    `VP_ENV_NAME=${name}`,
    `VP_RUNTIME_KIND=${runtime.kind}`,
    `VP_RUNTIME_VERSION=${runtime.version}`,
    `VP_NODE_START=${startFile || "index.js"}`, // arquivo de start do Node na 1ª subida
    `VP_PY_START=${startFile || "app.py"}`,
    `VP_PY_CMD=${base64(pythonCmd) | ""}`, // comando avançado Django/gunicorn (base64)
    `VP_DOTNET_CMD=${base64(dotnetCmd) | ""}`,
    `VP_PAGE_B64=${CONSTRUCTION_B64}`, // página "em construção" do placeholder
    `VP_PHP_ROOT=${phpRoot || "/var/www"}`,
    ...envVars // env REAL do Docker (do cliente), filtrando // RESERVED_ENV e prefixo VP_
  ],
  Labels: { "vp.env": envId },
  ExposedPorts: { "80/tcp": {} },
  HostConfig: {
    Memory: Math.round(limits.memMb * 1024 * 1024), // teto de RAM (bytes) — OOM-kill do kernel
    NanoCpus: Math.round(limits.vcpu * 1e9), // cota de CPU (CFS) — LIMITE REAL
    CpusetCpus: cpuset || undefined, // cores VISÍVEIS (cosmético p/ htop/nproc)
    RestartPolicy: { Name: "unless-stopped" }, // volta após crash/OOM/reboot
    Init: true, // tini como PID 1 (reap de zumbis, SIGTERM)
    Binds: binds.length ? binds : undefined, // LXCFS + veloz-code (se houver)
    PortBindings: { "80/tcp": [{ HostIp: "0.0.0.0", HostPort: "" }] }, // porta EFÊMERA no host
  },
  ... (attachNet ? { NetworkingConfig: { EndpointsConfig: {
    [network.name]: { IPAMConfig: { IPv4Address: ip } } } } : {}), // IP fixo na bridge do dono
}
```

Pós-criação (`docker.ts:694-724`):

- `start()`.

- `writeEnvFileAndRestart` — **materializa** `/veloz/env` (\$7).
- `waitForPort` (`docker.ts:948-960`) — **espera a :80 publicar** (20× 200 ms). Se não escutar → remove o container e devolve erro **"o app precisa escutar em 0.0.0.0:80"** com o tail do log.
- `readRuntimeVersion` — **grava** `runtimeVersionFull` (versão real).
- **Retorna** `{ containerId, httpPort, versionFull, phpNodeVersionFull }`.

A porta do app é sempre a :80 do container, publicada numa porta efêmera do host (`HostPort: ""` → o Docker escolhe). O agente lê a porta real via `inspect()` (`NetworkSettings.Ports["80/tcp"][0].HostPort`) e devolve como `httpPort`, que a API persiste em `environments.http_port`. É esse `httpPort` que o Caddy do control-plane usa como upstream (ver [02 \(02-ambiente-rede.md\)](#)).

2. Serviço (banco) — `provisionService()`

Diferenças-chave vs app: usa **imagem stock** (não a base de runtime), monta **volume de dados**, roda o **entrypoint nativo** da imagem (sem supervisor/Cmd), readiness por **exec** (não por porta), **não publica porta** no host (salvo painel embutido), e aplica **mais hardening**.

Ordem (`docker.ts:819-862`): `removeExistingByEnv` → `ensureImage` → `ensureNetwork` → **volume** `veloz-data-<envId>` no `dataPath` → **filtra env** → `pickCpuset` → `createContainer` → `start` → `waitReady`.

Objeto real (`docker.ts:837-860`)

```
{
  Image: args.image,                                // et.image, ex.: "rabbitmq:3-management"
  Env: env.length ? env : undefined,                  // credenciais do serviço (RABBITMQ_DEFAULT_USER...)
  Labels: { "vp.env": envId, "vp.role": role ?? "service" },
  ExposedPorts: pubKey ? { [pubKey]: {} } : undefined, // só se publishPort setado
  HostConfig: {
    Memory:    Math.round(memMb * 1024 * 1024),
    NanoCpus:  Math.round(vcpu * 1e9),
    CpusetCpus: cpuset || undefined,
    RestartPolicy: { Name: "unless-stopped" },
    Init: true,
    PidsLimit: 512,                                // teto de PIDs (anti fork-bomb) - cgroup
    CapDrop: ["NET_RAW", "NET_ADMIN"],              // mata ARP-spoof L2 na bridge compartilhada
    SecurityOpt: ["no-new-privileges"],              // sem escalada via binário setuid
    Binds: binds.length ? binds : undefined,         // veloz-data (se dataPath)
    PortBindings: pubKey ? { [pubKey]: [{ HostIp: "0.0.0.0", HostPort: "" }] } : undefined,
  },
  NetworkingConfig: { EndpointsConfig: { [network.name]: { IPAMConfig: { IPv4Address: ip } } } },
}
```

- `pubKey = publishPort ? "<publishPort>/tcp" : null`. Bancos puros = null (nenhuma porta no host; só alcançáveis pelo IP interno da bridge). **Exceção:** serviços com painel web embutido publicam essa porta — ex.: RabbitMQ publica a **15672** (a UI de management), enquanto a **5672** (AMQP) fica interna. Ver [painel de serviço \(./apps/api/src/service-panel.ts\)](#).
- **Readiness:** `waitReady(readiness)` (`docker.ts:784-792`) roda o comando via `sh -lc` até 40× (1 s de intervalo) até exit 0. Comandos por engine em `apps/api/src/services.ts`:
 - `redis: redis-cli ping | grep -q PONG`
 - `mysql/mariadb: mysqladmin ping -uroot -p"$MYSQL_ROOT_PASSWORD" ... alive`
 - `postgres: pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" -q`
 - `rabbitmq: rabbitmq-diagnostics -q ping`
 - `mongodb: mongosh --quiet --eval "db.adminCommand('ping').ok"`

⚠ **Assimetria de hardening (documentar!):** `PidsLimit`, `CapDrop` e `SecurityOpt` **estão só no container de serviço** (`provisionService`). O container de **app** (`provision`, `docker.ts:678-687`) **não aplica nenhum deles hoje** — só `Memory`, `NanoCpus`, `CpusetCpus`, `Init` e `RestartPolicy`. Se for **endurecer apps**, é aqui.

3. Stack (app + banco-filho) — `provisionStackEnv()`

`apps/api/src/provisioner.ts:116-162`. Um ambiente-stack tem **duas linhas** em `environments` (a raiz + a -db filha, ligada por `parentEnvId`) e **dois containers**:

1. **Banco-filho primeiro** (:128-146): `allocateAddress(childEnv.id, "service")`, credenciais próprias, `provisionService(role:"service")` sem `publishPort`. Grava container/estado do filho.
2. **App da stack** (:149-161): `allocateAddress(root.id, "app")`; reconstrói as credenciais reais do filho (`rebuildCreds`) e injeta no app via `stackAppEnv(...)` (ex.: `DB_POSTGRESDB_HOST/PORT/USER/PASSWORD` para o `n8n`, `services.ts:129-133`); `provisionService(role:"app", publishPort: et.internalPort, readiness: null)`. O app publica a porta web (`n8n=5678`, `wordpress=80`) e recebe `httpPort` → ganha subdomínio.

4. Resolução de imagem

`apps/agent/src/docker.ts:`

- `customImage(runtime) (:192-194):` `velozpanel/<kind>:<version>` — **a base rica** (`composer/extensões/git/toolchain/pnpm`).
- `officialImage(runtime) (:197-215):` **fallback cru** — `php:<v>-cli,python:<v>-slim,node:<v>-alpine,static → caddy:2-alpine,dotnet → mcr.microsoft.com/dotnet/sdk:<v>` (major 1-4 → `dotnet/core/sdk`).
- `resolveImage(runtime) (:222-230):` tenta inspect da base rica localmente; se existe usa, senão cai na oficial. Permite **rollout incremental** das bases por nó.
- `ensureImage(image) (:174-189):` inspect; se faltar, `docker.pull(followProgress)`.

Base images — `deploy/base-images/`

- `php.Dockerfile` — FROM `php:<v>-cli + composer 2 + git/unzip/zip/rsync/libs-dev + extensões pdo_mysql mysqli zip gd intl bcmath exif soap pcntl + PECL igbinary redis`.
- `node.Dockerfile` — FROM `node:<v>-slim` (Debian; evita `node-gyp/musl` do Alpine) + `git/curl/python3/build-essential/tini + npm + openssh`.
- `python.Dockerfile` — FROM `python:<v>-slim + git/build-essential/tini + gunicorn/whitenoise; alias python-pyhton3`.
- **Build:** `deploy/base-images/build-base.sh <php|node|python> <versões...> → docker build --build-arg <KIND>_VERSION=$V -t velozpanel/<kind>:$V`.

Versões suportadas — `packages/contracts/src/index.ts:21-27` (`RUNTIME_VERSIONS`)

```
php:      5.6, 7.0, 7.2, 7.3, 7.4, 8.0, 8.1, 8.2, 8.3, 8.4, 8.5
node:     18, 20, 22, 24, 25, 26
python:   3.8, 3.9, 3.10, 3.11, 3.12, 3.13, 3.14
static:   ["1"] (pseudovalor — estático não tem versão)
dotnet:   3.0, 3.1, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0
```

`RECOMMENDED_VERSION (:30-36):` `php 8.3, node 24, python 3.12, dotnet 8.0`. Enum `runtimeKind` em :14. **Adicionar um runtime/versão novo:** editar `RUNTIME_VERSIONS/runtimeKind` nos `contracts`, tratar o `kind` em `resolveImage/officialImage/cmdFor` (agente) e, se tiver base rica, buildar via `build-base.sh` e distribuir a imagem para os nós.

5. Volumes — o que sobrevive a quê

`ensureNamedVolume(name, envId) (docker.ts:761-767):` inspect; se faltar, cria com label `vp.env=<envId>`.

Volume	Quando	Conteúdo
<code>veloz-code-<envId></code>	<code>apps python / dotnet</code> (→/app) e static (→/site) o código buildado	
<code>veloz-data-<envId></code>	serviços com <code>dataPath</code> (<code>envTypes.dataPath</code>)	<code>datadir</code> do banco (ex.: <code>rabbitmq /var/lib/rabbitmq</code>)
<code>veloz-deploy-<envId></code>	container de build efêmero (→/workspace)	repo clonado, chave SSH, artefato (ver 03 (03-deploy.md))

Node e PHP NÃO têm volume de código — o código vive no `rootfs` do container e é a fonte-da-verdade o **git** (deploy).

Restart × recreate (durabilidade)

- **Reiniciar / pausar-iniciar / reboot do nó (mesmo container):** preserva **tudo** — `volume` ou não. `RestartPolicy: unless-stopped` faz o container voltar sozinho após `crash/OOM/reboot`.
- **Recriar o container (recreate)** — ex.: troca de versão de runtime, ou o toggle do painel que republica a porta: **só o que está em volume nomeado sobrevive**:
 - `Serviços` → dados em `veloz-data` ☐
 - `Python/.NET/Static` → código em `veloz-code` ☐
 - `Node/PHP` → código no `rootfs` ☐ (perde no `recreate`; restaurado pelo **deploy git**)
- Os volumes só são apagados no **delete job** (ver [04 \(04-ciclo-de-vida.md\)](#)).

6. LXCFs, cpuset e limites de recurso

- **LXCFS** (docker.ts:560-572, ligado por VP_LXCFS): bind-mounts de /var/lib/lxcfs/proc/ {cpuinfo, meminfo, stat, uptime, loadavg, diskstats, swaps} sobre /proc/*. Efeito: dentro do container, `htop/top/free/nproc` mostram os recursos do plano, não do host. Se o nó não tiver lxcfs instalado, deixar VP_LXCFS desligado.
- **cpuset** (pickCpuset, docker.ts:597-621, só com LXCFS): escolhe ceil (vcpu) cores (mín 1, máx = cores do host), os **menos usados** pelos outros containers `vp.env` (espalha carga). É **cosmético** (quantos cores o `nproc/htop` mostram); a **cota real de CPU continua sendo NanoCpus**.
- **Limites kernel-enforced** (não são cosméticos): `Memory` (cgroup memory → OOM-kill), `NanoCpus` (CFS quota → limite real de CPU), `PidsLimit` (cgroup pids, só serviços), `CapDrop/SecurityOpt` (capabilities/no-new-privileges, só serviços). Ver a nota sobre monitoramento (CPU% travado em 100, memória = working set) em `Plan//histórico`.

7. Supervisor, startup script e injeção de env

Apps de código não rodam o processo direto: o Cmd é um **supervisor shell** (cmdFor, docker.ts:265-374) que:

1. **startup_script (1ª subida)** — `setupPrefix` (docker.ts:241-249): grava o script do cliente em base64 (à prova de aspas) em `/veloz-startup.sh` e roda **uma vez**, guardado pelo marcador `/veloz-init-done`. Falha no script não derruba o app.
2. **carrega /veloz/env** — `LOAD_ENV` (docker.ts:260-263): lê linhas `KEY=base64(valor)`, corta só no 1º = (`k=${line%%=*}`; `v=${line#*=}`) e `export "$k=${printf %s "$v" | base64 -d}"`. **Nunca usa source/set -a** → injeção de shell é impossível (o valor vem base64).
3. **loop do processo** — relança o app (relando os arquivos de controle `/vp-node-start`, `/vp-python-cmd`, `/vp-php-root`, etc. e o `/veloz/env`) sempre que ele cai. O "restart" pós-deploy = matar o PID em `/vp-app-pid` (o supervisor relança) — **não** recria o container.

Injeção de variáveis de ambiente (duas vias)

1. **Env real do Docker** na 1ª subida (`Env[]` do `createContainer`), filtrando `RESERVED_ENV` (`PATH, LD_PRELOAD, LD_LIBRARY_PATH, NVM_DIR, HOME, PWD, SHELL, IFS, ENV, BASH_ENV, PS4`, docker.ts:254) e o prefixo `VP_`.
2. **/veloz/env** materializado por `writeEnvFileAndRestart` (docker.ts:397-408), `chmod 600`, **relido pelo supervisor a cada subida**. Marcador `/veloz-env-capable` indica que dá para **re-aplicar env ao vivo** (sem recriar). Se ausente (container legado), a resposta é `applied:false, reason:"recreate_required"`.

Resumo — app × service × stack

	app (provision)	service (provisionService)	stack
imagem	<code>resolveImage(runtime)</code>	<code>et.image stock</code>	<code>child.image + et.image</code>
Cmd	<code>supervisor cmdFor</code>	<code>entrypoint nativo</code>	<code>entrypoint nativo</code>
porta no host sempre	:80 (efêmera)	só se painel (<code>publishPort</code>)	app publica <code>et.internalPort</code>
volume	<code>veloz-code (py/dotnet/static)</code>	<code>veloz-data (dataPath)</code>	filho <code>veloz-data</code> ; app conforme tipo
hardening	<code>Memory/NanoCpus/Init/Cpuset + PidsLimit/CapDrop/no-new-privileges</code>	idem service	
readiness	<code>waitForPort :80</code>	<code>waitReady (exec)</code>	filho <code>exec</code> ; app <code>readiness null</code>
containers	1	1	2

Continua em: [02 — Ambiente na rede \(02-ambiente-rede.md\)](#) · [04 — Ciclo de vida \(04-ciclo-de-vida.md\)](#).

02 — Ambiente na rede

Como um ambiente ganha IP, fica isolado por dono, e é publicado com subdomínio + HTTPS.

Topologia

```
flowchart LR
    subgraph CP ["Control-plane (187) — hub WireGuard 10.100.0.1"]
        API["API :4000"]
        CADDY["Caddy do CP :80/:443"]
        PDNS["PowerDNS :53"]
        PG["Postgres"]
    end
    subgraph N1 ["Nó 184 — WG 10.100.0.4"]
        AG1["Agent :4100"]
        B1["bridge veloz-u<slot>: 10.201.<slot>:0/24"]
    end
    subgraph C1 ["container app :80"]
        app
    end
    subgraph N2 ["Nó local 111 — WG 10.100.0.3 (NAT)"]
        AG2["Agent :4100"]
    end
    API -- HTTP via WireGuard --> AG1
    AG1 -- HTTP via WireGuard --> AG2
    AG2 -- reverse_proxy wglp: httpPort --> C1
    C1 -- Internet --> jamees_top["jamees.top"]
    C1 -- 187 --> CADDY
    C1 -- B1
```

Três planos de rede distintos:

1. **Gestão (API → agente)**: por **WireGuard**. O agente escuta `:4100` **só no IP WG do nó** (`deploy/node/docker-compose.node.yml:29-31`), nunca público. O cliente não alcança o agente.
2. **Rede interna do ambiente**: cada container vive numa **bridge Docker por dono** (`veloz-u<slot>`), com IP fixo. Containers de donos diferentes ficam em bridges diferentes → isolados.
3. **Acesso público**: o Caddy do control-plane serve `<sub>.jamees.top` (HTTPS) e faz **reverse_proxy** para `IP-WG-do-nó:httpPort` por cima do túnel.

1. IPAM — alocação de IP (apps/api/src/ipam.ts)

Modelo

- **owner_networks** (schema.ts:324-340) — um /24 por (node_id, owner_id):
 - slot (3º octeto 0-255), subnet = 10.201.<slot>.0/24, gateway = 10.201.<slot>.1, bridge_name = veloz-u<slot>.
 - Únicos (node_id, owner_id) e (node_id, slot).
 - O 2º octeto é fixo 201 (ipam.ts:9-11) — escolhido pra não colidir com WG (10.77/10.100) nem docker0 (172.17).
- **env_addresses** (schema.ts:343-355) — um IP fixo por container: env_id, role (app | db | tool:<kind>), ip, container_id. Único (node_id, ip) → não-colisão por construção.

allocateAddress(nodeId, ownerId, envId, role) (ipam.ts:22-79)

- Roda em transação com **advisory lock por nó**: pg_advisory_xact_lock(hashtext('ipam:'+nodeId)) (ipam.ts:30) — serializa a alocação daquele nó (solto no COMMIT).
- **Idempotente**: se já existe IP para (env_id, role), **reusa** (ipam.ts:37-42).
- **Slot**: slot = coalesce(max(slot), -1) + 1 no nó (ipam.ts:45) — próximo livre sequencial; erro no_owner_subnet se > 255. Insere a owner_networks (ipam.ts:52-56).
- **IP**: varre hosts de .10 a .254 em 10.201.<slot>. (ipam.ts:64), pega o primeiro fora de env_addresses; erro subnet_full se cheio. .1 (gateway) e .2-.9 ficam reservados.
- Insere com on conflict (env_id, role) do nothing; trata corrida relendo o vencedor.
- Retorna { subnet, gateway, bridgeName, ip }.

Outros: ownerNetworkFor(nodeId, ownerId) (ipam.ts:82-91) lê a bridge existente (recriar container na mesma rede); releaseAddresses(envId) (ipam.ts:94-96) só apaga env_addresses (não remove a bridge — pode ter irmãos).

2. Bridge do dono no nó — ensureNetwork (apps/agent/src/docker.ts:735-758)

Idempotente (inspect; se existe, retorna). Cria via createNetwork:

```
{
  Driver: "bridge",
  CheckDuplicate: true,
  IPAM: { Driver: "default", Config: [{ Subnet: subnet, Gateway: gateway }] }, // 10.201.<slot>.0/24 / .1
  Options: {
    "com.docker.network.bridge.enable_icc": "true", // (!) ver divergência abaixo
    "com.docker.network.bridge.enable_ip_masquerade": "true", // egress p/ internet
  },
  Labels: { "vp.owner": ownerId },
}
```

O IP fixo é aplicado no createContainer via NetworkingConfig.EndpointsConfig[bridge].IPAMConfig.IPv4Address = ip. Migração a quente (dual-home) existe em attachNetwork (docker.ts:967-981).

⚠ **Divergência código × Plan (importante)**: o código cria a bridge com enable_icc=true e nome veloz-u<slot>. Os docs Plan/BANCO-POR-AMBIENTE.md descrevem icc=false, bridges brv<hash>, /16 por nó e enable_ip_masquerade=false — nada disso está no código. Ao documentar/mexer, use o comportamento real.

3. Isolamento multi-tenant — o que está no código × o que é config de host

No código (só containers de serviço, provisionService):

- CapDrop: ["NET_RAW", "NET_ADMIN"] — mata ARP-spoof L2 na bridge do dono.
- SecurityOpt: ["no-new-privileges"], PidsLimit: 512.
- Bridges por-dono distintas → o Docker já isola L2/L3 entre donos.

Lacunas / config manual de host (NÃO versionada — só em Plan/ como TODO):

- `icc:false` **global**, **chain** `DOCKER-USER` (drop inter-bridge no mesmo nó), `bridge-nf-call-iptables`, `sysctl` persistente → `Plan/SERVICOS-POR-AMBIENTE.md`;100-103, `Plan/BANCO-POR-AMBIENTE.md`;58-70, `Plan/REDE-DOCKER-VALIDACAO.md`;255-283.
- Anti-spoof inter-dono cross-node = `cryptokey` routing do WireGuard (AllowedIPs estrito) + `rp_filter`.
- Container de **app** não tem `CapDrop/no-new-privileges/PidsLimit` (só serviço tem) — ver [01 §2 \(01-criar-ambiente-docker.md\)](#).

Se for endurecer o isolamento de rede de verdade, o caminho é versionar `daemon.json` + a `chain DOCKER-USER` em `deploy/node/` (hoje é manual, e regras fora do `PostUp` do `wg0.conf` se perdem no `reboot` sem `iptables-persistent`).

4. WireGuard e resolução do nó (`apps/api/src/nodes.ts`, `agent.ts`)

- **Malha WG**: hub 187 = control-plane; nós rodam o agente. A API fala com o agente em `http://<wgIp>:4100`.
- **△ Duas faixas coexistem** (migração incompleta): `deploy/wireguard/README.md` e o `AGENT_URL/MariaDB` do `compose` usam **10.77.0.0/24** (hub .1, nós .2/.3); comentários de código e o PowerDNS do `compose` já usam **10.100.0.0/24** (`cp-ingress.ts`;31, `docker-compose.prod.yml`;73). **Na prática, o que vale é o `nodes.agent_url` gravado no banco** (em produção hoje é 10.100.0.x).
- `agentUrlForNode(nodeId)` (`nodes.ts`;15-19): lê `nodes.agent_url`; **fallback** `DEFAULT_AGENT_URL` (`agent.ts`;10, `AGENT_URL` ou `http://localhost:4100`).
- `pickNodeForNewEnv({region?})` (`nodes.ts`;31-77): **entre nós online com `agent_url`, filtra por região** (503 se sem capacidade) e escolhe o **menos carregado** (conta ambientes `running/paused/provisioning`).
- `httpHostForNode(nodeId)` (`nodes.ts`;91-95): `http_host ?? public_host`. Usado no `accessUrl` por IP:porta (nó NAT: `http_host` = IP LAN; `public_host` costuma ser só p/ SSH).

5. Publicação da porta HTTP

- **App**: `PortBindings`: { "80/tcp": [{ HostIp:"0.0.0.0", HostPort:"" }] } → **porta efêmera** do host, lida por `inspect` e devolvida como `httpPort` (persistido em `environments.http_port`).
- **Serviço**: sem publicação (só IP interno da bridge), **salvo** painel embutido (ex.: `rabbitmq` publica 15672).

A porta efêmera muda se o container for recriado (não num simples restart). Por isso, ao recriar/republicar, o `vhost` do Caddy precisa ser reescrito com a nova porta (é o que `syncSubVhost/enablePanel` fazem).

6. Ingress HTTP — Caddy do control-plane (`apps/api/src/cp-ingress.ts`)

A API escreve arquivos `*.caddy` em `CP_INGRESS_DIR` (default `/caddy-managed`), um **volume compartilhado** com o container do Caddy do CP, que faz `import /etc/caddy/managed/*.caddy` e **recarrega** ao detectar mudança (um `watcher` que dá `caddy reload` a cada ~5 s). TLS por-host é **HTTP-01 automático**.

- **Zonas**: `SUB_ZONE` = "jamees.top" (ambientes) e `TOOL_ZONE` = "jamees.top" (painéis de serviço) — `cp-ingress.ts`;12,17. `subFqdn(sub, zone)` → `${sub}.${zone}`.
- `wgIpFromAgentUrl(agentUrl)` (`cp-ingress.ts`;31-35): **extrai o host do `agent_url`** (`http://10.100.0.4:4100` → 10.100.0.4) → **é o IP WG do nó usado como upstream**.
- `putSite(sub, upstream, zone)` (`cp-ingress.ts`;38-44): **valida host + upstream (IP:porta) e escreve**:

```
<sub>.jamees.top {
    encode gzip zstd
    reverse_proxy <wgIp>:<httpPort>
}
```

- `removeSite(sub, zone)` (`cp-ingress.ts`;47-51): **apaga o `.caddy` (libera o nome, para de renovar o cert)**.
- **Quem escreve o upstream**: `syncSubVhost(sub, nodeId, httpPort)` (`environments.ts`;61-70) e o `backfill-subdomains.ts`.

DNS wildcard (`apps/api/src/dns-pdns.ts`, `db/backfill-subdomains.ts`)

- `*.jamees.top` A `<CP_IP>` (TTL 300) criado via `pdns.replaceRRsets` (`backfill-subdomains.ts`;20-28, `CP_PUBLIC_IP` default 187.127.49.205). Então **todo** `<sub>.jamees.top` resolve para o control-plane — criar um subdomínio novo **não** cria registro DNS, só escreve o `vhost`.

- O cliente do PowerDNS fala só pela HTTP API (PDNS_API_URL default `http://pdns:8081`), nunca no SQL. Nameservers `ns1.geestao.top=187 / ns2.geestao.top=184`.

⚠ **Use `jamees.top`, não `jamees.com`.** `jamees.com` é autoritativo no Cloudflare (sem wildcard `*.jamees.com`) → subdomínios não resolvem e o Let's Encrypt não emite. `jamees.top` é o PowerDNS de vocês, com wildcard funcionando. Pannel e ambiente compartilham a zona `jamees.top`; por isso `subdomain.isSubTaken` *cheça* `environments.auto_subdomain` e `env_tools.subdomain` (evita colisão).

7. `accessUrl` — como o painel decide a URL do ambiente

Server-side em `toEnvironment (apps/api/src/routes/environments.ts:75-91)`, por prioridade:

1. `r.domain` → `https://<domain>` (domínio próprio do cliente).
2. `r.autoSubdomain` → `https://<sub>.jamees.top`.
3. `r.httpPort` && `r.nodeId` && `!serviceUiPort (type)` → `http://<httpHost>:<httpPort>` (IP público/LAN do nó : porta efêmera).
4. serviço com painel embutido (rabbitmq): `panelUrl (...)` → `https://<sub>.jamees.top` do painel.

O painel só consome `env.accessUrl` (botão "Abrir site", linha "Principal" em Endereços).

Continua em: [01 — Docker \(01-criar-ambiente-docker.md\)](#) · [03 — Deploy \(03-deploy.md\)](#).

03 — Deploy do código do cliente

Como o código de um repositório git vira um app rodando. É diferente do **release da plataforma** (buildar/enviar as imagens `api/pannel/agent` — isso está em [05 \(05-release-plataforma.md\)](#)).

Princípio central (`apps/agent/src/deploy.ts:10-15`): o build roda num **container efêmero IRMÃO** (label `vp.role=build`, RAM própria de 2 GB, volume *veloz* — `deploy-<envId>`); depois os arquivos buildados são **COLOCADOS** (place) no container do app **já rodando**, sem recriá-lo. Quem orquestra é a API (`apps/api/src/routes/deploy.ts`); quem executa é o agente (`apps/agent/src/deploy.ts`).

flowchart TD; RUN["POST /deploy/run — cria deploy_run (running)"] --> AGENT["agent.startDeploy"]; AGENT --> BUILD["Fase BUILD (container irmão vp.role=build)"]; BUILD --> GIT["git_sync (clone/fetch shallow, SSH ou HTTP)"]; GIT --> STEPS["steps de build (composer/npm/pip/dotnet publish...)"]; STEPS --> OUT["monta /workspace/out + grava .veloz-sha"]; OUT --> PLACE["Fase PLACE (copia out → container do app)"]; PLACE --> APPSTEPS["app-steps (artisan migrate/optimize...) DENTRO do app"]; APPSTEPS --> RESTART["Fase RESTART (mata o PID; supervisor relança)"]; RESTART --> FIN["finalize(success, commitSha) + poda histórico"];

Marcadores no log ao vivo: `::vp:phase:* e ::vp:step:<kind>:start|exit:<rc>` (a UI parseia isso).

1. Fase BUILD (`apps/agent/src/deploy.ts:562-569`)

1. `startBuildContainer (:127-164)`: sobe o container efêmero. Imagem = `resolveBuildImage` (a base do runtime; fallback pra tag mais recente do mesmo tipo).
`Cmd: ["sh", "-c", "sleep 3600"], Memory = 2048 MB (BUILD_MEM_MB, :48), Binds: ["veloz-deploy-<envId>:/workspace"], WorkingDir: /workspace.`
2. `buildScript (args) (:354-465)` gera um shell script, transportado em **base64** e executado (`printf '%s' '<b64>' | base64 -d > build.sh && bash build.sh`). *Saída ao vivo por* `execStream (:518-531)`.
3. Se `exit != 0` → `finalize(failed)` e sai.
4. Lê o SHA do commit em `/workspace/out/.veloz-sha`.

git_sync — clone/checkout (: 359-364)

Sempre **shallow** (`--depth 1`):

```
if [ -d /workspace/repo/.git ]; then cd /workspace/repo && git fetch --depth 1 origin -- '<branch>' && git reset --hard FETCH_HEAD; else rm -rf /workspace/repo && git clone --depth 1 --branch '<branch>' -- '<repoUrl>' /workspace/repo; fi
```

Autenticação (no Env do container de build, :133-138):

- **SSH (deploy key)**: `GIT_SSH_COMMAND=ssh -i /workspace/.ssh/id_ed25519 -o BatchMode=yes -o StrictHostKeyChecking=accept-new -o UserKnownHostsFile=/workspace/.ssh/known_hosts`. A chave privada vive no volume (`/workspace/.ssh/`), gerada por `generateDeployKey (:197-221, ed25519)` ou importada (:167-190). Só a **fingerprint** é persistida no DB; a pública aparece 1× no painel. `known_hosts` via `ssh-keyscan github.com gitlab.com bitbucket.org`.
- **HTTP (user/senha ou token)**: injeta `GIT_USER/GIT_PASS` + credential helper (:158-162).

- **Imagens .NET SDK** não trazem `openssh-client` → instalado sob demanda (`guard command -v ssh`).

Detecção de framework — `detectStack` (:294–351)

O `kind` (runtime) vem do DB; a detecção refina o **framework**:

- **python**: `manage.py` → `django` (`suggestedPythonCmd = python manage.py runserver 0.0.0.0:80 --insecure --noreload`); **senão** `python + suggestedStartFile` (primeiro de `app.py` `main.py` `wsgi.py` `server.py`).
- **static**: `package.json` com `script build` → `spa`; **senão** `static`.
- **dotnet**: acha `*.csproj/*.*sln` → `dotnet`; **senão** `none`.
- **node** com `"next"`: no `package.json` → `nextjs` (`runModel=standalone`).
- `composer.json + artisan + laravel/framework` → `laravel`.

Steps de build por framework — `defaultSteps` (API, `routes/deploy.ts`:128–169)

- **Laravel**: `git_sync` → `composer_install` → (se `package.json`) `npm_ci` + `npm_build` → `laravel_fix_index` → `artisan_storage_link` → `artisan_clear` → `artisan_optimize` → `artisan_migrate` (desligado por padrão, `mutatesData=true`).
- **Python**: `git_sync` → (`requirements`) `pip_install` → `python_restart`.
- **Static**: `git_sync` → (`spa`) `npm_ci` + `npm_build`.
- **.NET**: `git_sync` → `dotnet_publish` → `dotnet_restart`.
- **Node/genérico**: `git_sync` → (`composer`) `composer_install` → (`package.json`) `npm_ci` + `npm_build` → `node_restart`.

Comandos concretos (`deploy.ts`:368–419):

- `composer_install`: `composer install --no-dev --optimize-autoloader --no-interaction --prefer-dist`.
- `npm_ci`: detecta `pnpm-lock/yarn.lock/package-lock` e usa o gerenciador certo (`pnpm install --frozen-lockfile / yarn --frozen-lockfile / npm ci`).
- `npm_build`: `nextjs` → `NEXT_PRIVATE_STANDALONE=true npm run build`; **senão** `npm run build --if-present`.
- `pip_install`: `pip install --no-cache-dir --target=.vp-vendor -r requirements.txt` (vendoriza em `.vp-vendor` → sobrevive a recreate).
- `dotnet_publish`: `dotnet publish "$PROJ" -c Release -o /workspace/publish -p:UseAppHost=false --nologo`.
- `shell`: comando livre do usuário.

Montagem do artefato → `/workspace/out` (:420–462)

- **Next.js standalone**: `.next/standalone/. + .next/static + public`.
- **Laravel**: `out/framework/` = repo inteiro (sem `.git`); `out/www/` = conteúdo de `public/`.
- **SPA**: auto-detecta `dist|build|out`; desce pra subpasta com `index.html` (caso Angular).
- **static/python/django**: `cp -a . out/`.
- **dotnet**: copia `/workspace/publish/..`.
- Grava `/workspace/out/.veloz-sha = git rev-parse HEAD`.

2. Fase PLACE — copia `out` para o container do app (:571–576)

Todos usam `getArchive` (build) → `putArchive` em `/veloz-incoming` (app) → `cp -a`. O `placer` varia por framework:

- `placeIntoApp` (:626–636) — **node/genérico**: `cp -a out/. <workdir>/` **sobrepõe, não apaga** (preserva `.env/uploads`).
- `placeStatic` (:642–654) — **deploy limpo**: `rm -rf tudo menos .vp-*` (preserva o `Caddyfile`) antes de copiar.
- `placeDotnet` (:661–673) — **deploy limpo** em `/app` (DLLs antigas não podem sobrar).
- `placeLaravel` (:682–704) — dois destinos no mesmo container:
 - `/var/projeto-laravel` (framework, **fora do docroot**): atualiza o código **preservando storage/ e .env**; garante dirs graváveis (`storage/...`, `bootstrap/cache`).
 - `/var/www` (docroot): substitui 100% com `out/www/.`; `symlink ln -sfn /var/projeto-laravel/storage/app/public /var/www/storage`.
 - Motivo: o `.env` fica **fora** do docroot → nunca é servido.

3. App-steps pós-place (:580–588)

Passos que precisam do ambiente do app (têm `.env/DB`), rodados via `execStream` (`appContainerId, ...`) — `appStepCommand` (`kind`) (:468–500). Prefixo `LOAD` carrega `/veloz/env` antes:

- `laravel_fix_index`: `sed` no `/var/www/index.php` **apontando pra** `/var/projeto-laravel/`.
- `artisan_storage_link, artisan_clear` (`optimize:clear`), `artisan_optimize` (`config/route/view:cache`), `artisan_migrate` (`migrate --force`).

4. Fase RESTART (:591–608)

Reinicia o **processo** do app com o código novo (o supervisor relança — ver [01.\\$7 \(01-criar-ambiente-docker.md\)](#)). Helpers em `docker.ts`: `applyNodeStart`, `applyPythonCmd/applyPythonStart`, `applyDotnetCmd/restartDotnet`, `applyPhpRoot (Laravel)`, `restartApp (static)`. Nenhum recia o container.

5. finally (:615–618)

`killContainer` remove o container de build; `writeLogToEnv` grava o log em `/veloz/deloys/<runId>.log` dentro do app, podendo pra `historyLimit` (default 10).

Modelo de dados (`apps/api/src/db/schema.ts:169–241`)

- **deploy_configs** (PK = `envId`): `connectionMode` (`none|public|ssh|local`), `provider`, `repoUrl`, `branch` (default `main`), `mode` (`simple|advanced`), `publicKey/fingerprint`, `httpUsername/httpPasswordEnc`, `deployStrategy` (`place|recreate`), `framework`, `runModel`, `subdir`, `historyLimit` (10; 0 = nunca apagar), `autoEnabled/intervalMinutes/nextCheckAt/lastRemoteSha` (auto-deploy, ver abaixo), `lastRunId/lastRunStatus/lastGoodSha`.
- **deploy_steps**: `ord`, `enabled`, `kind`, `command` (livre só em `kind="shell"`), `label`, `cwd`, `mutatesData`.
- **deploy_runs**: `trigger` (`manual|auto`), `status` (`running|success|failed|interrupted`), `exitCode`, `failedStepKind`, `commitSha/commitMessage/commitAuthor`, `stepsSnapshot` (jsonb), log (~64KB redigido), `startedAt/finishedAt`. Índice (`envId`, `startedAt`).

Rotas (API, `apps/api/src/routes/deploy.ts`, sob `/environments/:id/deploy`)

`GET /deploy` · `PUT /deploy/connection` · `POST /deploy/probe` · `POST /deploy/key/generate|import|test` · `PUT /deploy/http-credentials` · `PUT /deploy/history` · `GET/PUT /deploy/branch(es)` · `POST /deploy/steps/detect` · `PUT /deploy/steps` · `PUT /deploy/auto` · **`POST /deploy/run` (cria o run e chama `agent.startDeploy` — retorna já)** · `GET /deploy/runs` · `GET /deploy/runs/:runId/log` · `DELETE /deploy`.

`POST /deploy/run (:413–447)` monta o `buildEnv` só com as `env-vars` marcadas `buildTime=true`.

Logs de deploy — polling, não SSE

1. O agente guarda o log em memória (`deployState: Map<runId, ...>`, teto ~400KB), exposto por `GET /deploy/log/:runId` (agente), passando por `redact` (mascara `*KEY|TOKEN|SECRET|PASS*`, corta a 64KB).
2. A API (`GET /environments/:id/deploy/runs/:runId/log`) chama o agente enquanto `running`; quando termina, **persiste** no `deploy_runs` e **poda** o histórico pra `historyLimit`.
3. O painel (`.../deploy/[runId]/page.tsx`) faz **poll** a cada **1,5 s** enquanto `status===“running”`.

*SSE no projeto é só para **logs de container** (aba Logs) e **Redis pub/sub** — não para deploy.*

Auto-deploy por push — lacuna conhecida (roadmap)

- Não há **webhook** (nenhum handler `x-github/x-gitlab/push_event`).
- O polling está **parcialmente** implementado: `PUT /deploy/auto` grava `autoEnabled/intervalMinutes/nextCheckAt`, e existem as colunas `nextCheckAt/lastRemoteSha/lastCheckAt + deploy_runs.trigger="auto"`. Mas não há **scheduler** que consuma isso (o `worker.ts` só processa `provision_env/delete_env`; os crons de `billing/dns/metrics` não tocam `deploy`).
- Na prática, todo **deploy é manual** via `POST /deploy/run` (`trigger:"manual"` hardcoded).
- Para implementar: um cron que varra `deploy_configs` com `autoEnabled` e `nextCheckAt <= now()`, compare o SHA remoto (`git ls-remote`) com `lastRemoteSha` e dispare `startDeploy` com `trigger:"auto"`.

Continua em: [01 — Docker \(01-criar-ambiente-docker.md\)](#) · [05 — Release da plataforma \(05-release-plataforma.md\)](#).

04 — Ciclo de vida de um ambiente

Da chamada `POST /environments` até `running` (ou `error`), e a remoção. Isto é a **orquestração** que envolve a criação do container ([01 \(01-criar-ambiente-docker.md\)](#)) e a rede ([02 \(02-ambiente-rede.md\)](#)).

stateDiagram-v2 [*] --> provisioning: POST /environments provisioning --> running: provision OK provisioning --> error: PermanentJobError / tentativas esgotadas running --> paused: pausar paused --> running: iniciar running --> deleting: deletar paused --> deleting: deletar error --> deleting: deletar deleting --> [*]: runDeleteJob

1. POST /environments (apps/api/src/routes/environments.ts:197-317)

Entrada: createEnvironmentInput (packages/contracts/src/index.ts:576-587): name (2-40), plan (slug), runtime {kind,version} (obrigatório se app), type (slug em env_types; ausente = app), region?, nodeId?, template (default none; nextjs pré-configura).

Validações (em ordem):

Checagem	Erro	Linha
getPlan(plan) existe	400 invalid_plan	:208-209
plano ativo	400 plan_inactive	:212
limite de máquinas (não-admin): nº de envs raiz < plan.maxEnvironments	409 env_limit_reached	:215-228
type existe e ativo	400 invalid_type	:233-238
plano ≥ recursos mínimos do tipo (minVcpu/minMemMb)	422 invalid_plan_for_type	:242-256
app tem runtime	400 invalid_runtime	:258
saldo (não-admin): balanceCents ≥ ceil((plan+tipo)/720)	402 insufficient_balance	:263-274

Grava (sempre state="provisioning", nodeId=null — o nó é escolhido no worker):

- app (:280-293): runtimeKind/runtimeVersion; template nextjs seta nodeStartFile="server.js", insere deployConfigs (nextjs/standalone) e env-vars PORT=80/HOSTNAME=0.0.0.0/NODE_ENV=production.
- service (:294-299): typeId, runtimeKind="node".
- stack (:300-313): insere a raiz + uma 2ª linha <name>-db (typeId=child.id, parentEnvId=root.id), ambas provisioning.

Enfileira (:314): jobs.insert({ kind:"provision_env", envId: root.id, payload:{region, template} }). Retorna toEnvironment(root) na hora (provisionamento é assíncrono).

2. Fila de jobs (schema.ts:388-411) + Worker (apps/api/src/worker.ts)

jobs: kind (provision_env|delete_env), env_id (sem FK — delete apaga o env), status (queued|running|done|failed|canceled), attempts/max_attempts (provision=8, delete=20), run_after, locked_by/locked_at/heartbeat_at, last_error. Índices (run_after) e (env_id).

Worker (roda dentro do processo da API):

- CONCURRENCY = VP_WORKER_CONCURRENCY ?? 2, poll 2 s, pool Postgres dedicado.
- Reaper: re-enfileira jobs running com heartbeat_at < now()-90s. Poda: apaga done|canceled com finished_at < now()-7d.
- Claim (FOR UPDATE SKIP LOCKED, :42-54): pega 1 job queued com run_after <= now(), marca running, attempts+1.
- Advisory lock por env (:82-87): pg_try_advisory_lock(hashtextextended(envId,0)) na mesma conexão reservada. Se outro worker está no mesmo env → devolve o job com run_after = now()+5s. Garante ≤ 1 job por ambiente no cluster.
- Heartbeat: setInterval 20 s.
- Dispatch: provision_env → runProvisionJob; delete_env → runDeleteJob.
- Falha (:96-114): PermanentJobError ou attempts ≥ maxAttempts → status="failed" + environments.state="error". Senão retry com backoff: run_after = now() + min(300s, 2^attempts · 5s) + jitter.

3. Provisioner (apps/api/src/provisioner.ts)

runProvisionJob (:165-230), reconciliador idempotente:

1. Se já running → return.
2. nodeId nulo → pickNodeForNewEnv({region}) (menos carregado), grava.
3. agentUrl = agentUrlForEnv({nodeId}).
4. Roteia por envTypes.category: service → provisionServiceEnv, stack → provisionStackEnv, senão provisionApp.
5. Best-effort: subdomínio <sub>.jamees.top p/ envs web (putSite); e para serviços com painel embutido (rabbitmq), enablePanel(fresh).
6. Erro: PermanentJobError → state=error e re-lança (sem retry); transitório → grava errorMessage, mantém provisioning, re-lança (worker faz retry).

As três funções (provisionApp :30-59, provisionServiceEnv :62-91, provisionStackEnv :116-162) seguem o padrão: allocateAddress → (credenciais) → agent.provision(Service) → gravar containerId/httpPort/state=running. Detalhes de cada uma em [01 — Docker \(01-criar-ambiente-docker.md\)](#).

Helpers: envVarsFor (decifra env-vars), rebuildServiceEnv/rebuildCreds (reconstruem credenciais persistidas p/ o container casar com o painel), ensureServiceUiPublished (recria um serviço publicando a porta do painel).

4. Estados (contracts/index.ts:209-216)

provisioning | running | paused | error | deleting (coluna environments.state). Ações do painel: **pausar** (environments.ts → paused, cobrança suspensão), **iniciar** (→ running), **reiniciar** (restart do container), **deletar** (→ deleting + job).

5. Remoção — runDeleteJob (provisioner.ts:233-255)

Disparado por DELETE /environments/:id (environments.ts:535-538): marca raiz + filhos deleting, **cancela jobs** provision_env ainda queued, insere jobs{kind:"delete_env", maxAttempts:20}.

runDeleteJob, para cada [...filhos, raiz]:

1. cpIngress.removeSite(subdomínio).
2. Se containerId → agent.remove (confirma antes de apagar; falha → retry).
3. agent.removeVolume de veloz-data-<id>, veloz-deploy-<id>, veloz-code-<id> (best-effort).
4. releaseAddresses(id) (libera IPs no IPAM; a bridge fica).
5. db.delete(environments) por parentEnvId e por id.

No agente: removeExistingByEnv lista containers por label vp.env=<envId> e força remove.

Como adicionar um novo TIPO de serviço (checklist)

1. **Catálogo:** adicionar a linha em env_types no seed (apps/api/src/db/push-and-seed.ts): id(slug), category, image(stock), internalPort, dataPath, defaultTool?, priceMonthCents, minVcpu/minMemMb.
2. **Runtime do serviço:** um case em serviceRuntime (apps/api/src/services.ts) — env do Docker (credenciais), readiness, e o store (o que persiste cifrado).
3. **Conexão exibida:** um case em connectionInfo (services.ts) — host/porta/usuário/URL.
4. **Painel embutido (opcional):** se tiver UI web, adicionar em SERVICE_UI_PORTS (services.ts) e a exposição segue o modelo do [painel de serviço](#) (`./apps/api/src/service-panel.ts`).
5. **Distribuir a imagem** para os nós (pull automático no 1º uso, ou pré-carregar).

Continua em: [01 — Docker \(01-criar-ambiente-docker.md\)](#) · [03 — Deploy \(03-deploy.md\)](#).

05 — Release da plataforma (deploy do velozPanel em si)

Como publicar uma mudança de código da **plataforma** (api / painel / agente) em produção. Diferente do [deploy do cliente \(03-deploy.md\)](#) (git → app). Para instalar do zero, ver `deploy/README.md`.

Servidores (ver [[velozpanel-prod-deploy]])

Host	Papel	Notas
187.127.49.205	Control-plane: Postgres + MariaDB + API + Painel + Caddy + PowerDNS, via docker-compose em /opt/velozpanel/control-plane. Hub WireGuard.	Serve jamees.com (painel) e *.jamees.top (ambientes).
184.107.115.183	Nó de hospedagem (6 vCPU) e host de build das imagens, em /opt/velozpanel-src (sincronizado por rsync , não é um git clone).	
192.168.2.111	Nó de hospedagem local (atrás de NAT).	

Imagens: velozpanel/{api,painel,agent}:prod — buildadas no 184 e transferidas via `docker save | ssh docker load`.

⚠ **Schema:** não há migrations. O schema é aplicado por `push-and-seed.ts` (idempotente: `CREATE TABLE IF NOT EXISTS + ALTER ... ADD COLUMN IF NOT EXISTS`). Se a sua mudança tocou o schema (`apps/api/src/db/schema.ts`), **adicione o `ALTER` correspondente em `push-and-seed.ts` e rode o `db:push` no deploy.**

Fluxo (mudança só na API — o mais comum)

Do seu Mac (você tem SSH por chave para 184 e 187, e o 184 alcança o 187):

```
# 1. Commitar (num branch, se estiver na main)
git checkout -b feat/minha-mudanca && git add -A && git commit -m "..."
```

```
# 2. Sincronizar o fonte para o build host (184) — SEM --delete
rsync -az --exclude='.git' --exclude='node_modules' --exclude='.next' \
  --exclude='dist' --exclude='*.tar' --exclude='.turbo' \
  ./ root@184.107.115.183:/opt/velozplanel-src/
```

```
# 3. Buildar a imagem no 184 e transferir para o 187
ssh root@184.107.115.183 '
  cd /opt/velozplanel-src &&
  docker build -f apps/api/Dockerfile -t velozplanel/api:prod . &&
  docker save velozplanel/api:prod | ssh root@187.127.49.205 "docker load"
'
```

```
# 4. (Só se mudou o schema) aplicar o db:push com a imagem nova, ANTES de recriar
ssh root@187.127.49.205 '
  cd /opt/velozplanel/control-plane &&
  docker compose -f docker-compose.prod.yml --env-file .env run --rm --no-deps -T api \
    pnpm exec tsx src/db/push-and-seed.ts
'
```

```
# 5. Recriar a api (breve indisponibilidade de ~segundos)
ssh root@187.127.49.205 '
  cd /opt/velozplanel/control-plane &&
  docker compose -f docker-compose.prod.yml --env-file .env up -d --no-deps --force-recreate api
'
```

Mudou também o painel ou os contracts (contracts entram no bundle de api e painel): buildar e transferir `velozplanel/painel:prod` também (`docker build -f apps/painel/Dockerfile --build-arg NEXT_PUBLIC_API_URL=/api/v1 -t velozplanel/painel:prod .`) e recriar api e painel no passo 5.

Mudou o agente: buildar `velozplanel/agent:prod`, transferir para cada nó, e recriar o agente lá (`deploy/node/docker-compose.node.yml`), ou via `docker run` conforme `[[velozpanel-prod-deploy]]`.

Verificação pós-deploy

```
# containers saudáveis
ssh root@187.127.49.205 'cd /opt/velozplanel/control-plane && \
  docker compose -f docker-compose.prod.yml ps api painel'
```

```
# api respondendo (401/404 = ok; 502 = api caiu)
curl -sk -o /dev/null -w "%{http_code}\n" https://jamees.com/api/v1/session
```

```
# painel
curl -skI https://jamees.com/ | head -1
```

O `--force-recreate` causa ~segundos de indisponibilidade (o container reinicia). Se um cliente estiver usando o painel, ele vê alguns 502 transitórios até a api voltar (healthy).

Scripts de migração/backfill (rodar 1×)

Rodam com a imagem nova via `docker compose run --rm --no-deps -T api pnpm exec tsx src/db/<script>.ts`:

- `db/push-and-seed.ts` — `schema + seed` do admin (idempotente).
- `db/backfill-subdomains.ts` — wildcard `*.jamees.top` no PowerDNS + subdomínio `p/` envs web sem um.
- `db/backfill-panels.ts` — `liga/reescreve` o vhost dos painéis de serviço (`rabbitmq`) — reconciliador (respeita painel desligado pelo dono).

Rollback

As imagens antigas continuam no 187 até serem sobrescritas. Para voltar: recriar apontando para a tag/imagem anterior (se preservada), ou rebuildar do commit anterior e repetir o fluxo. `db:push` é **aditivo** (ADD COLUMN IF NOT EXISTS) — colunas novas não quebram a imagem antiga, então normalmente basta reverter a imagem.

Pegadinhas

- **rsync sem `--delete`** (o `/opt/velozpanel-src` do 184 não é git; `--delete` poderia apagar artefatos locais deles).
- **rsync do macOS** é antigo — não usar `--info=stats1` (use `--stats`).
- **NÃO** montar o `caddy_managed` errado: a api escreve os vhosts em `CP_INGRESS_DIR=/caddy-managed` e o Caddy lê o mesmo volume em `/etc/caddy/managed` (esse wiring existe em prod mas **não** está no compose commitado — não “consertar” removendo).
- O `.env` de produção fica **só no 187** (`/opt/velozpanel/control-plane/.env`) — nunca versionar.

Ver também: `deploy/README.md` (instalação do zero) e `[[velozpanel-prod-deploy]]` (detalhes de rede/WireGuard/SSH gateway).

Arquitetura do Jamees (velozPanel) — estado atual

Painel de hospedagem **multi-tenant** próprio. Marca visível: **Jamees** (roxo #634ca8). Identificadores internos/infra continuam `velozPanel/velozpanel/*` (não renomear).

Documento gerado a partir de um levantamento por especialistas (um por camada) sobre o código **como está hoje**. Diagrama: `docs/arquitetura.png`.

1. Visão geral

O Jamees é um “mini-cloud” que deixa o cliente criar ambientes (sites/apps e serviços de banco) que rodam em **containers Docker** espalhados por vários **nós de hospedagem**. Um **plano de controle** central orquestra tudo (banco de controle, API, painel, DNS, borda HTTPS) e fala com um **agente** em cada nó por uma malha privada **WireGuard**.

Dois grandes tipos de ambiente:

- **Código** (`category: app`): `static`, `node`, `php`, `python`, `dotnet` — cada um roda o app do cliente na porta 80 do seu container.
- **Serviço/Stack** (`service/stack`): `mysql`, `mariadb`, `postgres`, `mongodb`, `redis`, `rabbitmq`, e `stacks n8n/wordpress` (app + banco-filho).

2. Monorepo

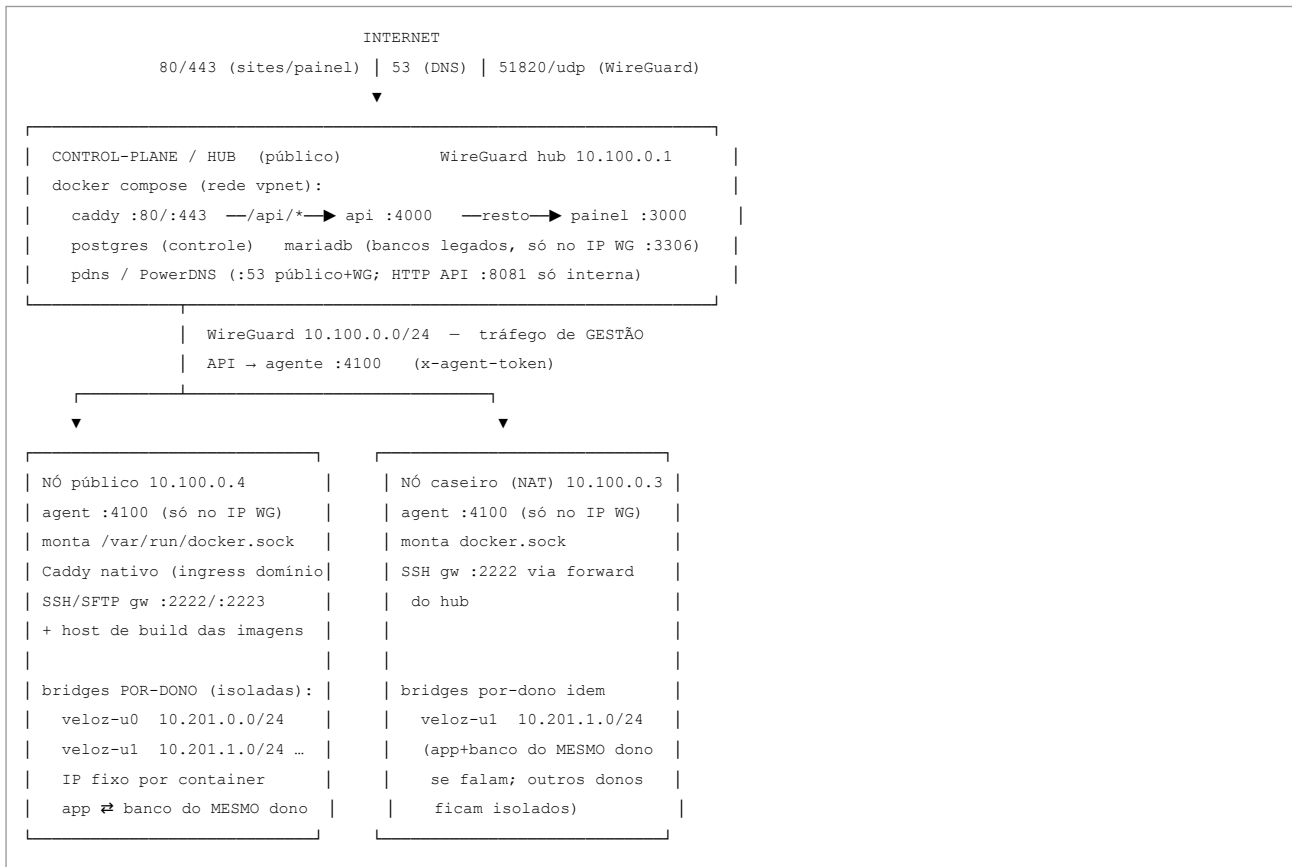
`pnpm workspaces` (Node ≥22), sem turbo — build/typecheck por pacote (`tsc --noEmit`), runtime via `tsx` (`api/agent`) e `next` (`painel`).

```
apps/
  api/      @velozpanel/api      - Fastify 5 + Drizzle (Postgres). O "cérebro".
  agent/    @velozpanel/agent    - Fastify 5 + dockerode. Um por NÓ; fala com o docker.sock.
  painel/   @velozpanel/painel   - Next.js 15 (App Router) + React Query. A UI.
packages/
  contracts/ @velozpanel/contracts - FONTE DA VERDADE (Zod): tipos e validação.
  db-console/ @velozpanel/db-console - motor do Jamees Studio (protocolo de banco).
```

- **contracts** é importado por **api + painel + agente**: os mesmos schemas Zod validam a request/response na API, tipam o painel e definem o “contrato de fio” com o agente. Os `Record<RuntimeKind, ...>` são exaustivos → adicionar um runtime novo quebra o typecheck até cobrir todos os mapas (foi o que garantiu o .NET completo).
- **db-console** é a lógica ÚNICA de banco do Studio (`classify` → `build` → `parse`), usada pela API (valida) e pelo agente (executa via `docker exec`).

3. Topologia de produção

Três servidores numa malha **WireGuard** 10.100.0.0/24 (o hub tem porta pública; os nós conectam nele, então nó atrás de NAT funciona sem porta de entrada).



- **Control-plane / hub:** servidor público. Postgres (banco de controle) + MariaDB (compartilhado, bancos "legados" dos clientes) + API + Painel + Caddy (borda 80/443) + PowerDNS autoritativo. É o hub WireGuard.
- **Nós de hospedagem:** rodam **só o agente**, que cria os containers dos clientes no `docker.sock` local. Um deles (público, mais parrudo) também é o **host de build** das imagens da plataforma. O nó caseiro está atrás de NAT e só sai pela WireGuard.

Observação: os templates versionados em `deploy/` usam sub-redes de exemplo (10.77.x); a malha em produção hoje é 10.100.0.0/24 (hub .1, nós .3/.4) e as bridges por dono são 10.201.<slot>.0/24.

4. As camadas

4.1 Painel (frontend) — apps/painel

- **Next.js 15 (App Router)**, React 19, Tailwind v4 (CSS-first), **React Query**, gráficos com uPlot, ícones lucide + PNGs de tecnologia (`public/img/tech`).
- Fala **direto** com a API (`http(s)://<host>:4000/api/v1`), sem proxy, com cookie de sessão `httpOnly vp_session` (`credentials: include`). Middleware redireciona a `/login` se não houver cookie.
- Dois grupos de rota:
 - **(app)** — cliente: lista de ambientes, `/env/[id]` com abas (Visão geral, Domínio & DNS, Configurações, Arquivos, SSL, SSH, SFTP, Deploy, Variáveis, **Data Studio**, Logs, Backups), `/dominios`, `/financeiro`. Guard `AuthGuard + AppShell`.
 - **(admin)** — super admin (`role=="admin"`): visão geral da frota, nós, usuários, ambientes de qualquer dono, rede (WireGuard), DNS global, **Planos e Preços**, Faturamento, auditoria. Guard `AdminAuthGuard + AdminShell`.
- Padrão de dados: `useQuery → lib/api.ts → API`; `mutations` invalidam as queries. Logs e Redis pub/sub usam **SSE**. Ambiente em `provisioning/deleting` faz polling (3s).

4.2 API / Control-plane — apps/api

- **Fastify 5 + Zod** (via `fastify-type-provider-zod`), **Drizzle** sobre **Postgres** (`postgres-js`), porta 4000, prefixo `/api/v1`.
- **Auth:** JWT em cookie `httpOnly` (`vp_session`), senhas `bcrypt`. **Segredos em repouso** cifrados com AES-256-GCM (`env vars`, credenciais de serviço, chaves de deploy).

- **Provisionamento assíncrono:** a API só enfileira `jobs` (`provision_env/delete_env`); um **worker** (fila persistente com `FOR UPDATE SKIP LOCKED` + `advisory lock` por `env`) reconcilia chamando o agente do nó. Escala horizontal "de graça" (cada réplica roda os loops; a fila evita colisão).
- **4 loops de fundo:** coletor de métricas (5s), cobrança (cheça a cada 30s), worker da fila, verificador de DNS (~5min).
- **IPAM** (`ipam.ts`): cada `(dono, nó)` recebe um `/24` (`10.201.<slot>.0/24`, `bridge` `veloz-u<slot>`); cada container ganha IP fixo. Livro-razão em `owner_networks / env_addresses`.
- **DNS:** gerencia o **PowerDNS** pela HTTP API (nunca escreve no SQL do `pdns`).
- **Ingress:** publica `<sub>.jamees.top` no Caddy do control-plane; domínios próprios do cliente vão no Caddy de cada nó (via agente).
- **Billing:** pró-rata por hora (preço do plano + adicional do tipo)/720; acerto ao deletar com **cortesia** configurável (`billingFreeMinutes`); rolup horário.

4.3 Agente do nó + Runtimes — apps/agent

- **Fastify + dockerode**, um por nó, escuta só no IP `WireGuard :4100`, toda rota exige `x-agent-token`. Fala com o `docker.sock` local. O controle chama o agente (o agente quase não faz chamadas de saída, exceto os gateways SSH/SFTP consultando a API interna).
- **Container do app** (`provision`): imagem por runtime, um **supervisor** shell (`cmdFor`) que sobe o app na `:80` num loop de auto-restart, injeta a página "**site em construção**" se não há código, aplica limites reais do kernel (`NanoCpus`, `Memory`, `CpusetCpus`), `LXCFS` (`htop/free` veem a cota), e nasce com **IP fixo na bridge do dono**.

Runtime	Imagem	Como roda na :80	Volume de código	Deploy
static	<code>caddy:2-alpine</code>	Caddy file-server (fallback SPA)	<code>veloz-code-*</code> → <code>/site</code>	<code>placeStatic</code>
node	<code>node:<v></code> (base própria)	<code>node <start></code> em loop	<code>— (/app)</code>	<code>placeIntoApp</code> + <code>start-file</code>
php	<code>php:<v>-cli</code> (base própria)	<code>php -S 0.0.0.0:80 -t <docroot></code>	<code>— (/var/www)</code>	Laravel: <code>placeLaravel</code>
python	<code>python:<v>-slim</code>	<code>python3 <start></code> ou cmd avançado (<code>gunicorn/Django</code>)	<code>veloz-code-*</code> → <code>/app</code>	<code>pip --target=.vp-vendor</code>
dotnet	<code>dotnet/sdk:<v></code>	cmd avançado → DLL publicada → <code>dotnet run</code> do exemplo	<code>veloz-code-*</code> → <code>/app</code>	<code>dotnet publish</code> → <code>placeDotnet</code>

- **Deploy por git** (`deploy.ts`): sobe um **container de build efêmero irmão** (volume `veloz-deploy-*`), clona, roda os passos (`npm`, `composer`, `pip`, `dotnet publish...`), monta o artefato e **copia** para o container do app sem recriá-lo. Passos que tocam o banco do dono (ex.: `dotnet ef database update`) **rodam com o build ligado à bridge do dono**. Chave de deploy é SSH (`ed25519`), gerada no volume.
- **Serviços** (`provisionService`): container por serviço na bridge do dono, IP fixo, volume `veloz-data-*`, **sem porta pública** para bancos, endurecido (`CapDrop NET_RAW/NET_ADMIN`, `no-new-privileges`, `PidsLimit`).
- **Gateways embutidos:** **SSH** (`:2222`, só por chave, `docker exec` no container do dono) e **SFTP** (`:2223`, só por senha, sem shell).
- **Monitoramento:** `/stats` devolve CPU% **relativo à cota** (travado em 100 — o kernel já impõe o teto) e **memória working set** (desconta cache). Logs por snapshot + SSE.
- **Jamees Studio:** `/db/exec` executa consultas via `docker exec` usando o `db-console`.

4.4 Packages compartilhados

- **contracts:** runtimes/versões/labels, planos, ambiente/tipo/categoria, deploy, billing, DNS, SSL/SSH/SFTP, admin, e o Jamees Studio. Um símbolo `Zod` é literalmente o mesmo contrato entre as três camadas → zero drift.
- **db-console:** `classify` (segurança: `statement` único, `whitelist` de verbos, `anti-injection`, `read-only` por engine), `build` (monta `ExecPlan` por engine; senha nunca interpolada — vem por `env` do container), `parse` (`TSV/CSV/EJSON/Redis`) e o wrapper estático do Mongo. Ponto único de evolução (trocar `docker exec` por um `sidecar` no futuro).

5. Fluxos principais

Criar ambiente — Painel `POST /environments` → API valida plano/tipo + saldo → grava `environments` (`state: provisioning`) + enfileira `provision_env` → **worker** escolhe o nó menos carregado, aloca IP (IPAM) e chama `agent.provision` → agente cria o container na bridge do dono, sobe o supervisor na `:80` → API grava `containerId/httpPort`, `state: running` e publica `<sub>.jamees.top` no Caddy.

Deploy — Painel conecta o repo (deploy key SSH) → `POST .../deploy/run` → API cria `deploy_run` e chama `agent.startDeploy` → agente sobe o **build container**, roda os passos e **coloca** o artefato no container do app → reinicia o app → log via SSE.

Cobrança — `cron` (a cada 30s, respeitando o intervalo) debita pró-rata cada ambiente `running/paused` em `credit_transactions`; **ao deletar**, faz o acerto do tempo usado com a cortesia de X minutos; se o saldo zera e `suspendOnZero`, **pausa os ambientes**.

Acesso do cliente ao site — 100% **público por domínio:** o Caddy do nó (ou do control-plane, para `<sub>.jamees.top`) faz `reverse_proxy` para o container, com HTTPS Let's Encrypt automático. Não depende da `WireGuard`.

6. Rede: GESTÃO vs PÚBLICO

- **Gestão** (privado, só WireGuard): API → agente (:4100), app → MariaDB compartilhado (10.100.0.1:3306), gateways SSH/SFTP consultando `/internal/*`. Portas 4100/3306 e os endpoints `/internal/*` **nunca** ficam públicos (o Caddy bloqueia `/internal/*` com 403).
 - **Público** (cliente final): Caddy em cada nó/control-plane serve os sites por domínio com HTTPS automático. O site do nó continua no ar mesmo se o control-plane cair.
-

7. Segurança & isolamento multi-tenant

- **Rede por dono**: cada dono tem bridges `veloz-u<slot>` isoladas; app e serviços do MESMO dono se falam (IP fixo), donos diferentes ficam separados. Bancos sem porta no host.
 - **Agente**: toda a API :4100 exige `x-agent-token` (defesa em profundidade).
 - **Segredos**: env vars e credenciais cifradas em repouso (AES-256-GCM); a senha do banco **nunca** sai do container (nem a API decifra para o Studio — o comando referencia a env).
 - **Serviços endurecidos**: CapDrop `NET_RAW/NET_ADMIN` (anti ARP-spoof na bridge), `no-new-privileges`, `PidsLimit`.
 - **Gateways**: SSH só por chave, SFTP só por senha, ambos confinados ao container do dono via `docker exec` (nunca shell no host).
-

8. Persistência (o que sobrevive a recreate)

- **Control-plane**: pgdata (controle), mariadata (bancos legados), `caddy_data/config`.
 - **Por ambiente** (nos nós): `veloz-code-*` (código de python/static/dotnet), `veloz-data-*` (dados de serviço/banco), `veloz-deploy-*` (workspace de build). O container do cliente é recriado na **mesma bridge/IP** (via `owner_networks/env_addresses`).
 - Config de MySQL: como não há `my.cnf` montado, o jeito durável ao recreate é `SET PERSIST/PERSIST_ONLY` (grava em `mysqld-auto.cnf`, dentro do volume de dados).
-

9. Modelo de dados (control-plane / Postgres)

Sem migrations — schema por `CREATE TABLE IF NOT EXISTS + seed` idempotente (`pnpm db:push`). Colunas de enum são text, validadas na borda pelos contratos.

- **Identidade/infra**: `users`, `nodes` (região, capacidade, `agentUrl` WG), `wg_peers`.
 - **Ambientes**: `environments` (central), `metric_samples`, `env_vars` (cifradas), `service_credentials` (cifradas), `env_tools`, `databases`.
 - **Catálogo (super admin)**: `plans` (recurso/preço), `env_types` (categoria/adicional/mín.), `reserved_subdomains`.
 - **Rede/IPAM**: `owner_networks`, `env_addresses`.
 - **Deploy**: `deploy_configs`, `deploy_steps`, `deploy_runs`.
 - **Acesso**: `ssl_configs`, `ssh_configs`, `ssh_keys`, `sftp_configs`.
 - **Fila/operação**: `jobs` (fila persistente), `audit_logs`.
 - **DNS**: `dns_zones_meta`, `env_domains` (o resto no DB pdns).
 - **Billing**: `credit_transactions`, `platform_settings`, `billing_run_hours`.
-

10. Pipeline de deploy das imagens

- Imagens da plataforma: `velozpanel/{api,painel,agent}:prod`, buildadas na raiz do monorepo no **nó de build** (mais parrudo, para não pesar no hub).
 - Transferência: `docker save | ssh <destino> 'docker load' (api/painel → control-plane; agent → cada nó)`, depois `docker compose up -d` no control-plane e `docker run` (agente) nos nós.
 - Imagens base ricas dos runtimes de cliente: `deploy/base-images/{php,node,python}.Dockerfile` (com `git/toolchain`) via `build-base.sh`. Runtime .NET usa a SDK oficial diretamente.
-

Marca visível: Jamees. Núcleo/infra: `velozPanel`. Este documento reflete o código em produção na data da geração; o diagrama visual está em `docs/arquitetura.png`.

Documentação de desenvolvedor — velozPanel / Jamees

Painel de hospedagem próprio: o cliente cria **ambientes** (apps de código, bancos/serviços, ou stacks), o sistema provisiona um **container Docker** por ambiente num dos **nós**, dá **rede isolada por dono, subdomínio + HTTPS** automáticos, e **deploy por git**. A cobrança é por hora.

△ **Fonte da verdade = o código** (*apps/, packages/, deploy/*). Os documentos em *Plan/* descrevem um *desenho-alvo* (aspiracional) e **divergem** do código em vários pontos (*bridge, isolamento de rede, faixa WireGuard*). Cada doc aqui marca essas divergências. Quando houver conflito, **o código manda**.

Índice

Doc	Assunto
Arquitetura (visão visual) (arquitetura.md) · diagrama (arquitetura.png)	Panorama da arquitetura com diagrama. Comece por aqui para o mapa geral.
01 — Criar ambiente (Docker) (01-criar-ambiente-docker.md)	Como o container de um ambiente é criado: imagem, volumes, limites kernel, hardening, portas, app × serviço × stack.
02 — Ambiente na rede (02-ambiente-rede.md)	WireGuard, bridge por dono, IPAM, isolamento multi-tenant, ingress Caddy + DNS.
03 — Deploy do código (03-deploy.md)	Pipeline de deploy (git → build efêmero → place → restart), presets por framework, logs.
04 — Ciclo de vida (04-ciclo-de-vida.md)	Rota de criação, fila de jobs, worker, provisioner, estados, remoção.
05 — Release da plataforma (05-release-plataforma.md)	Build/envio das imagens e recriação de api/painel em produção.

Monorepo (pnpm workspaces)

```
apps/  
  api/      Fastify + Drizzle (Postgres). Orquestra tudo, persiste, expõe /api/v1. Roda o WORKER da fila.  
  agent/    Roda em CADA nó. Fala com o Docker daemon (dockerode). Faz build/deploy, SSH gateway.  
  painel/   Next.js (App Router). O painel do cliente + super admin.  
packages/  
  contracts/ Zod: tipos/validação compartilhados (RUNTIME_VERSIONS, planos, todos os schemas de API).  
  db-console/ Console de banco embutido (Jamees Studio).  
deploy/  
  Artefatos de produção (docker-compose do control-plane e dos nós, base images, WireGuard).  
Plan/  
  Documentos de planejamento (aspiracionais – NÃO são a fonte da verdade).  
docs/  
  Esta documentação.
```

Os 4 processos

- **api** (*apps/api*, porta 4000) — recebe as chamadas do painel, valida com Zod (*packages/contracts*), grava no Postgres, e **enfileira** o trabalho pesado. Fala com o agente de cada nó por HTTP sobre WireGuard.
- **worker** (dentro do processo da api, *apps/api/src/worker.ts*) — consome a fila jobs (provisionar/remover ambiente) com `FOR UPDATE SKIP LOCKED + advisory lock` por ambiente.
- **agent** (*apps/agent*, porta 4100 só na WireGuard) — um por nó. Cria/remove containers, builda e faz deploy, expõe o gateway SSH (2222). O cliente **nunca** alcança o agente.
- **painel** (*apps/painel*, porta 3000) — Next.js atrás do Caddy do control-plane.

Fluxo ponta a ponta (criar um ambiente)

sequenceDiagram participant U as Painei (cliente) participant A as API (Fastify) participant DB as Postgres participant W as Worker (fila) participant P as Provisioner participant AG as Agent (no nó) participant D as Docker daemon participant CX as Caddy do CP
U->>A: POST /environments {name, plan, runtime/type}
A->>A: valida (plano, limite, saldo, recursos mínimos)
A->>DB: INSERT environments (state=provisioning)
A->>DB: INSERT jobs (kind=provision_env)
A->>U: 200 {environment} (assíncrono)
W->>DB: claim job (SKIP LOCKED + advisory lock por env)
W->>P: runProvisionJob
P->>P: pickNodeForNewEnv (menos carregado)
P->>P: allocateAddress (IP na bridge do dono)
P->>AG: provision / provisionService (HTTP via WireGuard)
AG->>D: ensureImage + ensureNetwork + createContainer + start
D->>AG: containerId + httpPort (porta efêmera do host)
AG->>P: {containerId, httpPort}
P->>DB: UPDATE environments (state=running, httpPort, containerId)
P->>CX: putSite(<sub>jamees.top → wglp:httpPort) (best-effort)
CX->>CX: recarrega, emite cert Let's Encrypt (HTTP-01)

Detalhes em [04 — Ciclo de vida \(04-ciclo-de-vida.md\)](#).

Modelo de dados (tabelas centrais)

O schema é criado por **SQL manual idempotente** em *apps/api/src/db/push-and-seed.ts* (`CREATE TABLE IF NOT EXISTS + ALTER ... ADD COLUMN IF NOT EXISTS`), **não** por migrations do Drizzle. O *schema.ts* é só a tipagem. Para aplicar mudanças de schema em produção: `pnpm --filter @velozpanel/api exec tsx src/db/push-and-seed.ts` (ou `docker compose run --rm --no-deps api pnpm db:push`).

Tabela

Papel

Arquivo

Tabela	Papel	Arquivo
environments	O ambiente (state, typeId, runtime, containerId, httpPort, autoSubdomain, domain, parentEnvId)	schema.ts:42+
env_types	Catálogo de tipos (php/node/.../rabbitmq/n8n): imagem, porta interna, dataPath, preço, mínimos	schema.ts:304-321
jobs	Fila (provision_env / delete_env)	schema.ts:388-411
owner_networks	/24 por (dono, nó): bridge veloz-u<slot>	schema.ts:324-340
env_addresses	Um IP fixo por container (role app/db/tool)	schema.ts:343-355
service_credentials	Credenciais de serviço, cifradas	schema.ts:375-383
env_tools	Painéis de UI por serviço (rabbitmq_mgmt, jstudio): enabled + subdomain	schema.ts:358-371
deploy_configs / deploy_steps / deploy_runs	Config e histórico de deploy	schema.ts:169-241
nodes	Os nós (agent_url na WG, public_host, http_host, region, status)	schema.ts:28+

Convenções de código (do dono)

- **Nunca** renomear identificadores internos: velozpanel/*, VP_*, @velozpanel/*, bridge veloz-u*, labels vp.env/vp.role. A **marca visível** é "Jamees"; o código/infra continua velozPanel.
- Subdomínios de ambiente e de painel de serviço usam **jamees.top** (wildcard no PowerDNS próprio). **Não** usar **jamees.com** para isso (é Cloudflare, sem wildcard). Ver [02 — Ambiente na rede \(02-ambiente-rede.md\)](#).
- PT-BR nos comentários e na UI.